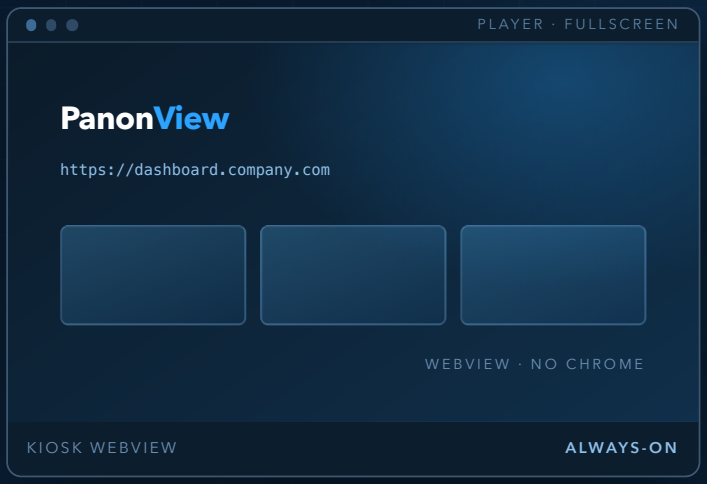
PanonView

PLAYER DATASHEET
RELEASE 1.0 · TAURI 2 EDITION
REV. 2026-11

EXTENDED CLIENT · PANONVIEW PLATFORM

PanonView Player



Boot straight into the screen.
Control it from anywhere.

The extended client app of the PanonView Platform: a cross-platform signage agent built with Tauri 2, Rust and React. It plays platform projects fullscreen and chrome-less, exposes a secured HTTP API on port 8787 for the central console, and keeps playing the last good page when the network is down.

0 UI
CHROME ON BOOT

8787
CONTROL API PORT

SQLite
DURABLE LOCAL STATE

24/7
UNATTENDED PLAYBACK

01 – BOOT
Player Mode
Launches fullscreen, borderless and straight into the configured URL. No address bar, no normal UI.

02 – CONTROL
HTTP API
An embedded Axum server on 0.0.0.0:8787 lets a CMS, ERP, dashboard or scheduler drive the screen.

03 – SCHEDULE
Time & Rotation
Time-based and rotation schedules with priorities, plus temporary override pages that auto-resume.

04 – RECOVER
Offline-First
The scheduler keeps running and the last successful page stays on screen instead of going blank.

01 Platform Overview

PanonView Player is the extended client app of the PanonView Platform – the edge agent that turns a TV, kiosk or video-wall PC into a managed display. It renders what the platform authors and publishes, keeps playing when the network drops, and exposes exactly the control surface the central console needs. Internally it combines a Tauri desktop shell, a Rust control core and a React admin surface in one small installable package.

WHAT THE PLAYER REPLACES

TYPICAL OBSTACLE	OPERATIONAL CONSEQUENCE
Browser left open manually	Address bar, updates and dialogs interrupt the screen
No remote control	Every change needs a site visit or remote desktop
Unauthenticated local API	Any LAN client can change what is displayed

PLAYER CAPABILITY	BUSINESS OUTCOME
Fullscreen player mode	Boots directly into content, kiosk-style
Time + rotation scheduling	Content changes automatically by day and time
Priority overrides	Emergency dashboards temporarily take over
API key + CIDR allowlist	Only approved clients can drive the player

DESIGN PRINCIPLES

One source of truth

The Rust DisplayController mediates every display change. HTTP API, scheduler and admin UI all route through it; nothing manipulates the WebView independently.

Offline-first by default

The scheduler keeps running locally. If a page fails, the player retains the last successful page instead of showing a blank screen.

Identity from day one

Device ID, name, location and group are stored in SQLite so the player can later join a central management platform without depending on it.

Remote-control ready

A real embedded Axum server, not a localhost asset plugin, exposes a documented REST API on port 8787.

Remembers the screen

Current URL, back/forward history and display state are persisted in SQLite and local storage, so a reboot returns to the same page.

Admin without distraction

The React admin UI is hidden at `#/admin`; normal boots bypass it and go straight to fullscreen playback.

Relationship to the PanonView Platform. The platform is the central console that designs, publishes, schedules and operates screens. The Player is the client that displays that output on site. The platform registers each player by IP, port and API key, then drives it through the player's REST API for status, live screenshots and remote actions – while the player continues scheduling and recovering entirely on its own if the console is unreachable.

02 Platform Architecture

A small, layered runtime: an experience layer for the screen and admin surfaces, a Rust control core that owns every display decision, a persistence layer for durable state, and an embedded HTTP edge for remote clients.

LOGICAL ARCHITECTURE

Experience & Access React + Tauri WebView

Player View
`fullscreen`
Borderless WebView showing the active page; no address bar or normal UI

Admin UI
`#/admin`
Dashboard, pages, schedules and settings; reachable in-app and via the web console

HTTP Clients
`:8787`
CMS, ERP, dashboard, scheduler or any REST client

▼

Control Core Rust · Tauri 2 · Tokio
DisplayController (single source of truth) · Axum API router · Scheduler tick · Tauri IPC commands · Auth middleware (API key / Bearer + CIDR)

▼

Persistence
SQLite with versioned migrations (`user_version`) · pages · schedules · display state · commands · settings · device identity

Runtime & Delivery
Tauri bundles for Windows, Linux, macOS · Android target · embedded UI bundle served at `/ui/` · unauthenticated `/health`

▼

External Systems
Remote web pages and dashboards · LAN browsers · REST automation · central management platform (future)

COMPONENT RESPONSIBILITIES

COMPONENT	PORT / SURFACE	RESPONSIBILITY
`DisplayController`	Rust core	Owns current URL, mode, history, overrides and every WebView change
`Scheduler`	Tokio task	Evaluates time and rotation schedules, priorities and override expiry
`Auth middleware`	API layer	API key / Bearer validation, CIDR allowlist and remote-control toggle

Why a real server. The control API is an embedded Axum server, not Tauri's localhost plugin. That keeps the API reachable on the LAN, documented like a normal REST service, and independent of app asset serving.

03 PanonView Platform Integration

The Player is the extended client app of the PanonView Platform. The platform is the authoring and operations console – design, publish, schedule and fleet management. The player is the edge runtime that renders the result on the physical screen and reports back what it is showing.

DIVISION OF RESPONSIBILITY

CONCERN	PANONVIEW PLATFORM – CENTRAL	PANONVIEW PLAYER – EDGE
Content	No-code dashboard designer on a resolution-locked canvas	Renders the published viewer or any web URL in a fullscreen WebView
Distribution	Publishes projects plus viewer and slideshow URLs	Loads the target URL and keeps it on screen
Time rules	Owns platform-wide scheduling and priority policy	Enforces local TIME and ROTATION schedules plus overrides
Fleet state	Registers players and shows live status and screenshots	Reports device identity, platform, version and current URL
Recovery	Re-issues commands when a player becomes reachable	Keeps the last good page and reschedules autonomously

INTEGRATION CONTRACT – PLATFORM ACTION TO PLAYER ENDPOINT

PLATFORM ACTION	PLAYER ENDPOINT	BEHAVIOUR
Register player	`GET /device · GET /status`	Reads device id, name, location, group, hostname, platform and version
Refresh display	`POST /display/refresh`	Reloads the current page without losing its place
Reset to default	`POST /display/reset`	Returns to the scheduled or default page
Live screenshot	`GET /display/screenshot`	PNG snapshot of what the screen is showing right now

Connection model

The platform reaches each player over the LAN at `http://:8787/api/v1` using the API key registered for that player. No inbound tunnel or extra service on the server is required.

Viewer URL targeting

A published project is opened through the platform viewer, for example `/view?id=…&mode=player&apiKey=…`, so the player shows exactly the designed canvas and resolves its data bindings.

Graceful degradation. The player never depends on the platform to stay on air. It boots, schedules, overrides and recovers locally; when the console returns, status, screenshots and remote actions resume over the same API with no site visit.

Fleet-ready client | Register by IP and API key | Live screenshots | Remote actions | Bulk operations | Status and version sync | Autonomous offline

04 Player Capabilities

Everything needed to run an unattended screen: boot behavior, scheduling, overrides, persistence, administrative access and a documented remote-control surface.

CORE CAPABILITIES

01
Player mode
Boots fullscreen, chrome-less and straight to the configured URL. Designed for kiosk and signage displays.

02
Embedded HTTP API
Axum server on 0.0.0.0:8787 for remote control from a REST API, ERP, CMS, dashboard or scheduler.

03
SQLite persistence
Versioned migrations keep pages, schedules, display state, commands, settings and device identity across upgrades.

04
Scheduling
Time-based schedules by day and time window, plus rotation schedules that cycle pages by duration.

05
Priority overrides
Emergency or temporary pages supersede the scheduler and auto-resume when they expire.

06
Offline-first playback
The scheduler keeps running locally and the last successful page remains on screen instead of going blank.

07
Last-page memory
Current page/URL and back/forward history persist in SQLite and local storage, surviving stop, crash or reboot.

08
Hidden admin UI
React console at `#/admin`, plus a LAN web console at `/ui/` for headless players.

09
Secure by default
API key / Bearer auth, CIDR IP allowlist and a remote-control toggle; a key is generated on first boot.

SCHEDULING PRECEDENCE

ORDER	RULE	BEHAVIOR
1	Active override	Highest priority until its duration expires, then the scheduler resumes automatically
2	Active TIME schedule	The matching time window with the highest `priority` wins
3	ROTATION schedules	Pages cycle in sequence, each shown for its configured duration
4	Last successful page	Offline fallback when no schedule or page can be resolved
5	First enabled page	Final fallback for a fresh or empty schedule set

05 Control API & Services

A small REST surface under `/api/v1`. The same controller serves the HTTP API, the scheduler and the admin UI, so every path produces consistent state.

AUTHENTICATION

API key / Bearer

Send `X-API-Key: <key>` or `Authorization: Bearer <key>`. The key is generated on first boot and shown under Settings → API Key.

LAN access controls

CIDR IP allowlisting (for example `192.168.1.0/24`), plus an Allow Remote Control toggle for non-loopback clients.

ENDPOINT MAP

GROUP	ENDPOINTS	PURPOSE
Display	`GET /display · POST /display · POST /display/page · POST /display/override · POST /display/refresh · POST /display/back · POST /display/forward · POST /display/reset · GET /display/screenshot`	Read or change the active page, push a temporary override, navigate history, capture a screenshot
Pages	`GET /pages · GET /pages/:id · POST /pages · PUT /pages/:id · DELETE /pages/:id`	CRUD for saved pages and URLs
Schedules	`GET /schedules · GET /schedules/:id · POST /schedules · PUT /schedules/:id · DELETE /schedules/:id`	CRUD for time and rotation schedules
Settings	`GET /settings · PUT /settings`	Read or update player settings such as the HTTP port
Device	`GET /device · PUT /device`	Read or set name, location and group
System	`GET /status · GET /commands · POST /restart · POST /shutdown · POST /reload`	Health, command history and lifecycle actions
Probe	`GET /health`	Unauthenticated liveness check

EXAMPLE REQUESTS

```
# Show a page immediately
curl -X POST http://192.168.1.20:8787/api/v1/display \
  -H "X-API-Key: $PV_KEY" \
  -H "Content-Type: application/json" \
  -d '{"url":"https://dashboard.company.com"}'
```

```
# Emergency override for 5 minutes
curl -X POST http://192.168.1.20:8787/api/v1/display/override \
  -H "X-API-Key: $PV_KEY" \
  -H "Content-Type: application/json" \
  -d '{"url":"https://dashboard.company.com/alert","duration":300,"priority":100}'
```

06 Security, Operations & Use Cases

Designed for screens that are physically remote but must remain trustworthy, recoverable and easy to operate.

SECURITY CONTROLS

CONTROL	IMPLEMENTATION
Authentication	API key and Bearer token enforced for all HTTP clients, loopback included
Network allowlist	CIDR-based client filtering, for example `192.168.1.0/24, 10.10.0.0/16`
Remote-control switch	Allow Remote Control toggle applies to non-loopback clients
Liveness probe	`GET /health` remains unauthenticated for monitoring only

PLATFORM NOTES

PLATFORM	BEHAVIOR
Windows / Linux / macOS	Native fullscreen, always-on-top and borderless window; shared Rust core and HTTP API
Android / Android TV	Same codebase; device-test WebView behavior, immersive fullscreen, D-pad focus, autostart and kiosk mode

REPRESENTATIVE USE CASES

Corporate signage

Lobby dashboards, meeting-room status, internal announcements and KPI screens driven by an existing CMS.

Retail & hospitality

Menu boards, promotions and queue displays that switch by time of day and recover from network loss.

Operations & industry

Factory, warehouse or control-room screens with priority emergency overrides and offline fallback.

Education

Campus wayfinding, classroom displays and event schedules managed over the LAN.

Healthcare

Waiting-room information and staff dashboards where the last good page must stay visible.

Managed fleets

Device identity and a documented API provide a clean path to a future central management platform.

PanonView

PanonView Player – the platform's dependable screen, controlled from anywhere.

Extended client app of the PanonView Platform · Tauri 2 + Rust + React · Fullscreen WebView · Embedded REST API · SQLite persistence · Time and rotation scheduling · Priority overrides · Offline-first playback · API key and CIDR security.

DOCUMENT **PLAYER DATASHEET** RELEASE **1.0** LICENSE **MIT** REV. **2026-11**